

CS 663 : Digital Image Processing : Assignment 2

Instructor : Suyash P. Awate

Note: The input data/images for a question are present in the corresponding “data/” subfolder.

Use of any generative-AI based tools (aiding plagiarism) is prohibited. If any such usage is found, it will be treated as a case of plagiarism. The difficulty level for all assignments in this course, including all underlying aspects of pre-processing, the main algorithm implementation, post-processing, and visualization is low enough that the use of sophisticated coding tools isn't required. Needless to say, the goal of the assignment is to learn the image-processing algorithms, and NOT to learn to engineer appropriate prompts to get solutions from generative-AI tools.

5 points are reserved for submission in the described format.

When displaying grayscale images, use a colormap that uses at least 200 colors/intensities, and display the colormap/colorbar alongside that image.

Unless visualizing RGB images, don't round the resulting images to integers; use a floating-point data type for the pixel value.

1. (15 points) Image Sharpening.

Input image 1: `data/sharpen/moon.png`

Input image 2: `data/sharpen/peacock.png`

Input image 3: `data/sharpen/tiger.png`

(a) (10 points)

Write a function to implement unsharp masking for sharpening the edges in the input images. Clip the intensities in the output image if they go beyond the range of the intensities in the input image.

(b) (5 points)

Display the original image along with two versions of the sharpened image obtained with two different levels of sharpening that are clearly perceivable. Display all images on the same colorscale as the input image.

2. (20 points) Template Matching.

Input image 1 (scene): `data/templateMatch/parking.png`

Input image 2 (template): `data/templateMatch/templateNoPark.png`

(a) (5 points)

Implement a function to search for a match for the template image (templateNoPark) within the scene image (parking) using normalized cross correlation (NCC) for each color channel.

(b) (5 points)

Use the function to compute the NCC images for 3 different template image sizes, where (i) the size of the scene image is reduced by a factor of 5 in each dimension, and (ii) the size of the template image is resized to 41×41 , 51×51 , and 61×61 . Display the NCC images computed for each color channel, in each of the 3 aforementioned cases.

(c) (5 points)

The given template image has a white background just outside the red ring/annulus, but the parking sign in the scene image has a blue background just outside the red ring/annulus. Implement another NCC-based template-matching function that computes NCC (between the template and each image patch) by excluding the region outside the red ring/annulus in the template image.

(d) (5 points)

Use the modified function to compute the NCC images for 3 different template image sizes, where the size of the template image is resized to 201×201 , 251×251 , and 301×301 . Display the NCC images computed for each channel, in each of the 3 aforementioned cases.

3. (10 points) Canny Edge Detection.

Input image 1: data/edge/butterfly.png

Input image 2: data/edge/paithaniEdge.png

Input image 3: data/edge/rangoli.png

(a) (5 points)

Implement a function for canny edge detection. Tune the free parameters to get a reasonable output where all major edges for all major objects in the image are detected.

(b) (5 points)

Display the input image and output image (binary) showing the detected edges, side by side.

Also display an image that shows the detected edge pixels drawn, in black color, on the the input image itself.

4. (20 points) Edge and Corner Detection.

Input image 1: `data/corner/nandadevi.png`

Input image 2: `data/corner/paithaniCorner.png`

Input image 3: `data/corner/warli.png`

(a) (5 points)

Implement a function to compute the structure tensor at each pixel in the image, and then to compute the eigenvalues and eigenvectors of each structure tensor.

(b) (5 points)

Implement a function for Harris-Stephens edge detection, Harris-Stephens corner detection, and Shi-Tomasi corner detection. Tune the free parameters independently for all methods to get a reasonable output where all major corners for all major objects in the image are detected.

(c) (10 points)

Display the

(1) input image,

(2) image of the per-pixel largest eigenvalue of the structure tensor,

(3) image of the per-pixel second-largest eigenvalue (Shi-Tomasi corner-ness) of the structure tensor,

(4) image of the per-pixel Harris-Stephens corner-ness measure, say C ,

(5) images of the per-pixel Harris-Stephens and Shi-Tomasi corner-ness measure after non-maximum suppression,

(6) binary images of Harris-Stephens and Shi-Tomasi corner-detection outputs,

(7) images that show the Harris-Stephens's and Shi-Tomasi's detected corner pixels drawn, in black color, on the input image itself.

(8) image of the per-pixel Harris-Stephens edge-ness measure (C) after non-maximum suppression,

(9) binary image showing Harris-Stephens edge-detection output,

(10) image that shows the Harris-Stephens's detected edge pixels drawn, in white color, on the input image itself.

5. (20 points) Image Segmentation using Mean Shift.

Input image 1: `data/segment/ghats.png`

Input image 2: `data/segment/leopard.png`

Input image 3: `data/segment/bison.png`

(a) (5 points)

Implement a function to perform mean-shift segmentation using pixel features that are a combination of pixel's RGB color and the pixel's XY coordinates. Tune all free parameters so that the method outputs a few segments (not more than 10 or so) per image.

(b) (5 points)

Implement a function that takes the output of the mean-shift method and creates a label image (segmentation map) where pixel values range from 1 to N, each indicating a segment out of a total of N segments. You may want to make your code computationally efficient by approximately modeling the feature-distribution using a Gaussian-kernel based nonparametric density using only a subset of pixels.

Implement a function that takes two inputs, i.e., the RGB image and its label image (segmentation map), and creates an image that shows the segment boundaries (make them 2-pixel thick; black colored) drawn on the input image itself.

(c) (5 points)

Display the input image along with its segmentation (binary image) side by side.

Also display another image where the segment boundaries are drawn on the input image itself.

6. (20 points) Bokeh Effect.

Input image 1: `data/bokeh/deep.png`

Input image 2: `data/bokeh/lotus.png`

Input image 3: `data/bokeh/marigold.png`

(a) (5 points)

Use the function written earlier (based on the mean-shift algorithm) to segment the input image into two parts: the main “object” (flowers in `lotus.png` and `marigold.png`) and the remaining background. If need be, you may perform some simple pre-processing/post-processing before/after the segmentation function. Your output should be a binary image indicating the two segments.

For `deep.png`, you can consider the metal vessel and all its contents as the main object. You may define an outline manually, instead of performing automatic segmentation.

(b) (10 points)

Implement a function to blur the background using a Bokeh/disc-shaped filter of diameter 50 pixels. To handle boundary conditions, at the image boundary as well as at the object-background boundary, the filter should be cropped and its values rescaled.

Redo the filtering with a Bokeh/disc-shaped filter of diameter 100 pixels.

Make your code computationally efficient by processing pixels close to the boundaries (where filter gets cropped) differently from the pixels away from the boundary (where filter doesn't get cropped). [You may use a distance transform for this purpose.]

(c) (5 points)

Display the original image along with two filtered images. Display all images on the same colorscale as the input image.